

Let's Encrypt HTTPS — Design Document

Revision: 1 **Last modified:** 2026-07-01T11:00:00Z **Status:** Draft — design scout deliverable, NOT yet implemented. Awaiting the OPERATOR INPUT REQUIRED decisions below before Phase 0 can close. **Authority:** Inherits the Helix Constitution submodule (constitution/Constitution.md) per §11.4.35. Honors §11.4.161 (rootless Podman), §11.4.76 (containers submodule), §11.4.156 (no CI/CD, no git hooks), §11.4.10 / §11.4.30 (secrets), §11.4.74 (catalogue-first), §11.4.99 / §11.4.150 (latest-source research), §11.4.169 (all test types), CONST-033 (no host power changes). **Companion plan:** LETSENCRYPT_HTTPS_PLAN.md

This is a DESIGN, not an implementation. No source code, no container boot, no host network changes were made producing it. It is written to be handed to implementers, who execute the phased plan in the companion doc.

1. Current TLS / certificate reality (Step 1 findings — cited)

Everything below was read directly from the working tree. File:line citations are exact.

1.1 The HTTP/SOCKS forward proxies do NOT terminate TLS

- **Squid** (docker-compose.yml:56-120, config config/squid/squid.conf) is a **forward caching proxy** on HTTP_PROXY_PORT (default 53128, .env.example:16). It does **not** present a server certificate. The only TLS-related lines in config/squid/squid.conf are standard CONNECT-tunnel ACLs:
 - config/squid/squid.conf:33 — acl SSL_ports port 443
 - config/squid/squid.conf:36 — acl Safe_ports port 443
 - config/squid/squid.conf:50 — http_access deny CONNECT ! SSL_ports

There is **no https_port, no sslcrtd, no ssl_bump**. Client HTTPS is *tunnelled* through the CONNECT verb, never decrypted. SSL inspection / MITM is explicitly disabled: `.env.example:269-271` → `HTTPS_INSPECTION=false` ("WARNING: Requires certificate setup"). So Squid needs **no** public certificate today.

- **Dante** (`docker-compose.yml:127-194`, config `config/dante/sockd.conf`) is **SOCKS5** on `SOCKS_PROXY_PORT` (default 51080). SOCKS5 is not a TLS protocol; no server cert.

Conclusion: the proxy data-plane is **client-configured forward proxying**. Clients point their browser/OS at `HOST_IP:53128` / `:51080`. There is no server hostname a browser validates, hence **no public LE cert applies to Squid/Dante**.

1.2 The control-plane API uses INTERNAL mTLS with a PRIVATE client CA — out of scope for public LE

- `control-plane/internal/api/tls.go:25-56` — `buildTLSConfig` sets `cfgTLS.ClientAuth = tls.RequireAndVerifyClientCert (tls.go:54)`. This is **mutual TLS**: every caller must present a client certificate signed by a configured **private client CA**.
- Key material is loaded from **file paths that are Podman-secret mount points**, never embedded (`tls.go:8-11`, `.env.example:144-158`):
 - `CONTROL_API_TLS_CERT=/run/secrets/helixproxy_api_cert` (secret name `helixproxy_api_cert`)
 - `CONTROL_API_TLS_KEY=/run/secrets/helixproxy_api_key`
 - `CONTROL_API_TLS_CLIENT_CA=/run/secrets/helixproxy_api_client_ca`
- The server (`control-plane/internal/api/server.go:169-212`) serves REST CRUD + SSE (`/events`) + Prometheus `/metrics` + PAC (`/proxy.pac`) over that mTLS socket on `CONTROL_API_ADDR` (default `:58080`, `.env.example:142`). It is **fail-closed**: no valid client identity ⇒ no access (`tls.go:4-6`).

Why LE does not fit here. Let's Encrypt is a public **Domain-Validation CA** that issues **server** certificates only. It does not, and cannot, issue a **client CA** for an internal mTLS trust boundary. The control-plane API is an internal service reached by internal callers holding internal client certs; substituting a public LE server cert would neither remove the need for the private client CA nor pass any public ACME challenge (there is no public DNS name or reachable validation path for an internal service). **The mTLS API is therefore explicitly OUT OF SCOPE for this**

LE work and stays exactly as designed. LE and mTLS are orthogonal: one authenticates the *server* to public browsers, the other authenticates *clients* to a private server.

- Optional plaintext /metrics listener: control-plane/internal/api/server.go:126-158 (startMetricsListener), config CONTROL_API_METRICS_ADDR (.env.example:170, **empty/OFF by default**). Plaintext, pod-internal, for a non-mTLS Prometheus scraper. Not a public surface, not a candidate for LE.

1.3 The only web/HTTPS-shaped surface is the Caddy admin interface — and it is plaintext :80, not even wired into compose

- config/caddy/Caddyfile:3 — :80 { ... }. **Plaintext port 80**. It is a static file server (root * /srv, file_server browse), a /health responder, a /status JSON page, and a stubbed /api/* → 501 (Caddyfile:1-42). It fronts services/admin/index.html.
- **Caddy is NOT a running service today**. A search of both docker-compose.yml and docker-compose.dynamic.yml for a caddy service returns nothing — the Caddyfile is a **config artifact only**. The compose "admin" that *does* run is proxy-admin = traefik/whoami on host :58080 plaintext (docker-compose.yml:199-211).

1.4 Compose services + exposed ports (inventory)

Service	File:line	Port(s) exposed	TLS today
proxy-vpn (OpenVPN client gateway)	docker-compose.yml:11-49	53128, 51080	none (tunnel egress)
proxy-squid* (HTTP forward proxy)	docker-compose.yml:56-120 / docker-compose.dynamic.yml:289-326	53128	none (CONNECT tunnel)
proxy-dante* (SOCKS5)	docker-compose.yml:127-194	51080	none
proxy-admin (traefik/whoami)	docker-compose.yml:199-211	58080 (host net)	none (plaintext)
proxy-gluetun (dynamic)	docker-compose.dynamic.yml:128-	internal 8888	none

Service	File:line	Port(s) exposed	TLS today
VPN egress)			
proxy-redis / proxy-postgres	docker-compose.dynamic.yml:67- / 89-	internal	none
control-plane API (cmd/api)	control-plane/internal/api/server.go	58080	internal mTLS (private CA)
Caddy admin	config/caddy/Caddyfile (unwired)	80 (plaintext)	none

1.5 Plain statement of the actual public HTTPS surface

There is NO existing public-facing HTTPS surface in helix_proxy today. Nothing terminates a public server-cert TLS handshake. Therefore:

Let's Encrypt does not "add HTTPS to an existing endpoint" here — it requires INTRODUCING a TLS-termination front. The natural, lowest-friction front is the **Caddy admin/dashboard**, which already exists as a Caddyfile (config/caddy/Caddyfile) and whose engine (Caddy) has best-in-class automatic HTTPS. This design upgrades that plaintext :80 admin surface into a Caddy-terminated public HTTPS surface and wires it in as a rootless, containers-submodule-orchestrated service. The forward proxies and the internal mTLS API are untouched.

What the public LE cert is *for*: the **operator-chosen dashboard hostname** (see OPERATOR INPUT REQUIRED §9). Example only: proxy.example.com.

2. Catalogue-check result (§11.4.74)

Checked both organizations for a reusable cert/acme/letsencrypt/tls/proxy-front module, via authenticated gh (GitHub) and glab (GitLab):

```
gh repo list vasic-digital | grep -iE 'cert|acme|letsencrypt|
tls|ssl|caddy|traefik|proxy'
→ vasic-digital/helix_transport (HelixVPN Rust transport:
MASQUE/Hysteria2/Shadowsocks/UDP-over-TLS)
```

```
gh repo list HelixDevelopment | grep -iE 'cert|acme|letsencrypt|
tls|...'
  → HelixDevelopment/HelixProxy (this repo), HelixDevelopment/
HelixGitpx (git proxy – unrelated)
glab repo list --group vasic-digital | grep -iE 'cert|acme|
tls|...' → no match
glab api groups/vasic-digital/projects?
search=cert          → []
```

Verdict: no-match. There is **no** existing owned ACME/cert/TLS-termination submodule to reuse. `helix_transport`'s "UDP-over-TLS" is a VPN *transport* obfuscation layer, not an ACME/cert-issuance or HTTPS-termination component — not applicable.

Catalogue-Check line for the tracker: Catalogue-Check: no-match vasic-digital + HelixDevelopment (no cert/acme/tls submodule); reuse the published Caddy engine via the containers submodule per §11.4.76.

Reuse decision: rather than author a bespoke ACME client, this design **reuses the Caddy server engine** (an off-the-shelf, published artifact — not a new in-house implementation) and **orchestrates it through the existing submodules/containers module** (§11.4.76), which already provides `pkg/boot`, `pkg/compose`, `pkg/health`, `pkg/crossbuild`, `pkg/volume`, `pkg/orchestrator`. No new org submodule is created. If, in future, multiple projects need the same Caddy-ACME front, the `extract-to-submodule` path (§11.4.74 "extend") is a follow-up — noted, not required now.

3. Deep research summary (Step 2 — cited, accessed 2026-07-01)

3.1 ACME client trade-offs

Client	Language	Renewal model	Reload	DNS-01 providers	Fit for this project
Caddy (automatic HTTPS)	Go	built-in in-process loop — no external timer/cron/hook	hot in-memory swap, zero-downtime	90+ via CertMagic/libdns (custom build)	BEST — already present as a Caddyfile; renewal loop satisfies §11.4.156 with no cron/CI/hook; is <i>also</i> the

Client	Language	Renewal model	Reload	DNS-01 providers	Fit for this project
lego	Go	issue/ renew command — needs an external timer	writes files → downstream must reload	90+	TLS terminator so no separate reload plumbing Good as a library if we ever embed ACME in Go, but the control- plane is internal- mTLS and must NOT become a public terminator; as a CLI it still needs a systemd -- user timer (more moving parts)
certbot	Python	certbot renew — ships its own systemd timer	writes files → deploy- hook reload	many (plugins)	Heavier (Python/ snap), timer- based; no advantage here
acme.sh	POSIX shell	cron- installed by default	writes files → reloadcmd	150+	Minimal deps but its default renewal is cron , which collides with the spirit of \$11.4.156's "host mechanism, not cron-in- CI" unless

Client	Language	Renewal model	Reload	DNS-01 providers	Fit for this project
					re-homed to a systemd timer
Traefik ACME	Go	built-in loop	hot reload	many	Equivalent to Caddy but we already have a Caddyfile; introducing Traefik duplicates the front

Sources: [Lego docs](#), [Caddy Automatic HTTPS](#), [ACME clients on Ubuntu \(OneUptime, 2026-03\)](#), [certbot vs acme.sh \(SSLInsights, 2026\)](#).

Recommendation: Caddy automatic HTTPS. Rationale: (a) already present in the repo as a Caddyfile — catalogue-first reuse; (b) the ACME client *and* the TLS terminator are the same long-running process, so renewal → hot-reload is one built-in mechanism with **zero downtime and no SIGHUP/restart** (Caddy docs: "Caddy keeps all managed certificates renewed"); (c) its renewal is an **in-process goroutine loop inside a long-running container** — exactly the "long-running renewal container/loop" the constitution's §11.4.156 discussion allows, needing **no cron, no CI pipeline, no git hook**; (d) DNS-01 + HTTP-01 + TLS-ALPN-01 all supported.

3.2 Challenge type for THIS host/topology

The host runs mission-critical parallel workloads; binding public :80/:443 and exposing an inbound validation path is a risk that must be an explicit operator choice.

Challenge	Needs inbound	Wildcards	Behind NAT	Notes
HTTP-01	port 80 externally reachable	no	no	simplest, but requires opening inbound :80 on the mission-critical host
		no	no	

Challenge	Needs inbound	Wildcards	Behind NAT	Notes
TLS-ALPN-01	port 443 externally reachable			avoids :80, still needs inbound :443
DNS-01	nothing inbound (writes a TXT record via DNS provider API)	yes	yes	needs a DNS provider API token; validation is outbound-only

Sources: [Caddy Automatic HTTPS](#) ("HTTP challenge requires port 80... TLS-ALPN requires port 443... DNS challenge does not require any open ports"); [Rootless Podman ACME/DNS](#) ([Caddy Community](#)) ("Rootless Podman won't be able to open the default HTTP and HTTPS ports (80 and 443)"); [wildcard TLS with Caddy + Cloudflare, 2026](#) ([HostMyCode](#)).

Recommendation: DNS-01. Rationale: (a) it needs **no inbound port** for *validation* — the safest option for a mission-critical host that should not casually accept inbound :80; (b) it works behind NAT; (c) it supports **wildcard** certs (*.proxy.example.com) if the dashboard grows subdomains; (d) it sidesteps the rootless-Podman privileged-port problem for the *challenge* (note: serving the resulting HTTPS still needs :443 reachable to clients — see §4.4). The cost is a **DNS provider API token**, handled as a Podman secret (§7). **HTTP-01 is the fallback** if the operator confirms inbound :80 can be safely bound and prefers not to manage a DNS token — this is an operator decision (§9).

3.3 Renewal, rotation, reload, staging, rate limits

- **Renewal loop.** Caddy/CertMagic renews automatically on an internal schedule (commonly when $\sim\frac{1}{3}$ of the lifetime remains). The exact day count is a CertMagic default, not pinned in the fetched Caddy page — the design does **not** hard-code a number; it treats "Caddy renews well before expiry" as the mechanism and *verifies* it in the renewal-simulation test (§6). [Caddy docs](#): "Caddy keeps all managed certificates renewed."
- **Rotation / reload with zero downtime.** Caddy swaps the renewed cert **in memory** on the live listener — no SIGHUP, no container restart, no dropped connections. This is the decisive advantage over file-writing clients (lego/certbot/acme.sh), which need a downstream reload hook.
- **Staging vs production ACME.** Always issue against **Let's Encrypt STAGING** (<https://acme-staging-v02.api.letsencrypt.org/directory>) first — untrusted chain but

same protocol, generous limits — then cut to production only after staging is green. [LE Staging Environment](#).

- **Rate limits.** Production LE: **50 certificates per registered domain / 7 days** (new issuance); **5 duplicate certificates / week**; **renewals are exempt** from the per-domain limit and, when signalled via **ARI (ACME Renewal Info)**, exempt from rate limits generally. [Rate Limits, Shorter lifetimes & rate limits, 2026-02-24](#). Implication: use staging for ALL iteration; production issuance is a one-shot cutover; renewals never hit the limit.
- **Shrinking lifetimes.** LE is moving default lifetime **90 → 64 → 45 days** over two years; renewal frequency will roughly double. [LE 2026-02-24](#). This is **why a self-driving in-process renewal loop (Caddy) is strategically correct** — the more frequent the renewals, the more valuable a zero-touch renewer. Any timer-based approach must shorten its interval as lifetimes shrink; Caddy adapts automatically (ARI-aware).

3.4 Hermetic / anti-bluff testing building blocks

- **Pebble** — Let's Encrypt's tiny in-RAM RFC-8555 ACME test server; boots in seconds, no persistence, used in the official test suites of lego/certbot/getssl. Ideal for **hermetic** integration tests: issue a real cert against a local ACME with zero network / zero rate-limit exposure. [Pebble, How Pebble supports ACME client developers \(LE, 2025-04-30\)](#).
 - **step-ca** — a real private ACME server (persistent) for a longer-lived internal ACME endpoint if ever needed. [Run your own private ACME server \(Smallstep\)](#). Pebble is preferred for CI-shaped hermetic tests; step-ca is the heavier alternative.
-

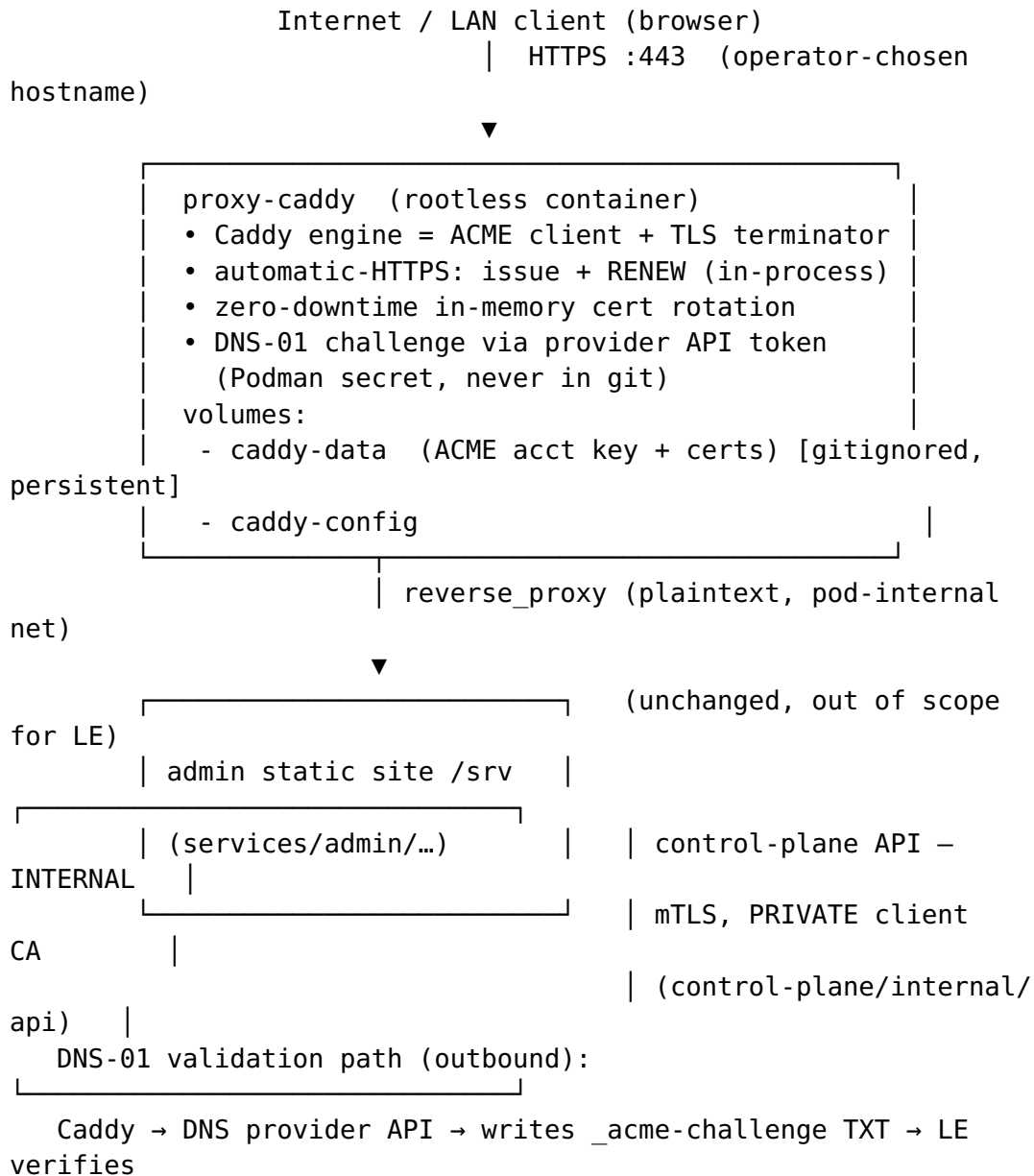
4. Recommended architecture

4.1 One-paragraph summary

Add a **rootless Caddy container** (proxy-caddy), orchestrated by the **containers submodule** (§11.4.76), that terminates public HTTPS for the operator-chosen dashboard hostname and reverse-proxies to the existing admin static site (and, if the operator wants, a read-only status endpoint). Caddy obtains and renews the certificate via **ACME DNS-01** (recommended) using a **DNS-provider API token supplied as a Podman secret**, storing the ACME account key + issued certs in a **persistent, gitignored volume**. Renewal and rotation are **built into the long-running Caddy process** (no cron, no CI, no git hook — §11.4.156), with

zero-downtime in-memory cert swap. Staging LE is used for all iteration; production is a gated one-shot cutover. The internal mTLS control-plane API and the forward proxies are unchanged.

4.2 Component diagram (textual)



4.3 Where it plugs in — exact services/ports/files

- **New compose service proxy-caddy** in a NEW overlay docker-compose.https.yml (kept out of the pristine base docker-compose.yml per the project's §11.4.122 overlay pattern, mirroring how docker-compose.dynamic.yml overlays the base). It publishes :443 (and optionally :80 for HTTP→HTTPS redirect only). It joins a dedicated pod-internal Podman network so it can reverse_proxy the admin site by service name (per the Caddy-Community rootless guidance: put Caddy + proxied

containers on a dedicated Podman DNS-enabled network rather than host port-forwarding).

- **config/caddy/Caddyfile** — upgrade the `:80 { ... }` block to a **site block keyed by the operator hostname** with a `tls` directive selecting the ACME issuer (DNS-01 provider + email + CA endpoint from env), retaining the `/health`, `/status`, `file_server` behaviour behind HTTPS, with an automatic `:80→:443` redirect.
- **Custom Caddy image.** The official `docker.io/library/caddy` image does **not** bundle DNS-provider modules ([oneuptime, 2026-02-08](#)). A **custom image** built with `xcaddy` + the chosen `caddy-dns/<provider>` module is required for DNS-01. Build it through the containers submodule **pkg/crossbuild** and a new `config/caddy/Containerfile` (mirrors the existing `config/squid/Containerfile.dynamic` pattern). For HTTP-01 the stock image suffices (no custom build) — a real reason the challenge choice affects Phase 1.
- **Orchestration** via the containers submodule: a `pkg/boot` + `pkg/compose` entry brings proxy-caddy up on demand, `pkg/health` gates readiness on a TLS handshake + `/health 200`. No ad-hoc `podman run` (§11.4.76). Rootless (§11.4.161).
- **.env.example** — new NON-SECRET vars only (values in git are placeholders; the token itself is a **secret name**, never a value):
 - `CADDY_DASHBOARD_DOMAIN=` (operator sets; e.g. `proxy.example.com`)
 - `CADDY_ACME_EMAIL=` (operator sets; LE account contact)
 - `CADDY_ACME_CA=https://acme-staging-v02.api.letsencrypt.org/directory` (staging default; prod is the cutover change)
 - `CADDY_ACME_CHALLENGE=dns` (or `http`)
 - `CADDY_DNS_PROVIDER=` (e.g. `cloudflare`, only for DNS-01)
 - `CADDY_DNS_TOKEN_SECRET=helixproxy_caddy_dns_token` (Podman **secret name**, never the value — §11.4.10)
 - `CADDY_HTTPS_PORT=443` / `CADDY_HTTP_PORT=80`
- **Untouched:** `control-plane/internal/api/*` (internal mTLS), `config/squid/*`, `config/dante/*`, all proxy data-plane. LE adds a surface; it removes nothing (§11.4.122).

4.4 Rootless-Podman port note (design constraint, not a host change here)

Rootless Podman cannot bind privileged ports (<1024) by default ([Caddy Community](#)). Serving public HTTPS still requires `:443` to reach the container. Options for the *serving* port (an operator/host decision, **not performed by this design** — CONST-033 and "no host network changes" respected):

1. Set `net.ipv4.ip_unprivileged_port_start=443` on the host (a `sysctl` the **operator** applies; documented in the rollout runbook, not executed here).
2. Publish the container on a high port (e.g. `:8443`) and front it with an operator-managed firewall/NAT redirect `443→8443`.

3. Run behind an existing host reverse-proxy / load-balancer that already owns :443.

DNS-01 keeps the *challenge* free of any inbound-port requirement; only the *serving* port question remains, and it is enumerated for the operator (§9). The design assumes option (1) or (2) and does not depend on which.

5. Security (§11.4.10 / §11.4.30)

- **ACME account key:** generated by Caddy on first run, stored in the caddy-data volume. That volume path is **gitignored** (§11.4.30) and declared with a §11.4.77 regeneration note (re-issue via ACME on loss). The account key is a secret — never committed, never printed in logs.
 - **DNS provider API token** (DNS-01 only): a **Podman secret** (podman secret create helixproxy_caddy_dns_token ..., created by the operator OUT of band). Only the **secret name** (CADDY_DNS_TOKEN_SECRET) appears in .env.example / compose; the value never touches git or .env (§11.4.10). It is mounted to Caddy as an env var sourced from /run/secrets/... or injected via the compose secrets: stanza. Scope the token to **DNS-edit on the single zone** only (least privilege).
 - **Pre-store leak audit (§11.4.10.A):** before the operator stores the token, run the repo-wide audit (git ls-files | xargs grep -l <value>, git log -S<value> --all) to confirm no prior leak of that specific value.
 - **Issued cert + private key:** in caddy-data, gitignored, 0600-class perms inside the container. Never committed (§11.4.30). The public cert MAY be surfaced read-only for evidence, the private key NEVER.
 - **.gitignore additions:** caddy-data/, caddy-config/, any *.pem/*.key under a Caddy data path, plus the standard secret patterns already covered.
 - **Blast radius:** the Caddy front is a *new* internet-facing surface. It exposes ONLY the admin static site + /health + /status (read-only) — never the mTLS control API, never the proxy ports. Keep the reverse-proxy allow-list tight; do not proxy /api/* mutating paths through the public front.
-

6. Test strategy across all §11.4.169 types (anti-bluff)

Every PASS cites captured evidence via `ab_pass_with_evidence` under `qa-results/<run-id>/letsencrypt/...`. Feature class per §11.4.69: `network_connectivity / tls` for the handshake, plus a project-local `acme_cert` class.

Type (§11.4.169)	What it proves	Mechanism / evidence
unit	cert parsing + expiry math (days-to-expiry, "is renewal due?" boundary), Caddyfile template render, env→issuer mapping	Go/table tests on a synthetic cert; golden-good + golden-bad analyzer fixtures per §11.4.107(10). Mocks allowed HERE ONLY (§11.4.27).
integration (hermetic)	Caddy issues a REAL cert against a local ACME (Pebble) with DNS-01 solved locally	boot Pebble via the containers submodule ; assert Caddy serves a cert whose chain roots in Pebble's test CA; capture the served leaf + chain. No network, no rate limits.
integration (staging)	end-to-end against LE STAGING real endpoint	real DNS-01 TXT written+cleared; assert a staging-issued (untrusted-but-valid-shape) cert for the real hostname; capture openssl <code>s_client</code> chain.
e2e	real client → real HTTPS → admin / health 200 over the served cert	drive a real HTTPS GET; capture status + served cert SANs.
renewal simulation	the renewal loop fires near expiry	seed a fake near-expiry cert into <code>caddy-data</code> ; observe Caddy renew it against Pebble/staging; capture before/after <code>NotAfter</code> . §11.4.135 guard.
rotation	old cert → new cert, service serves the NEW chain with zero downtime	force renewal; hold a live connection across the swap; assert the served leaf serial changed AND no connection dropped.
negative	expired / invalid / self-signed cert is	present an expired cert → client rejects; remove the

All non-unit tests use the **containers submodule** to boot Pebble/step-ca/Caddy (§11.4.76); topology-absent ⇒ honest SKIP-with-reason (§11.4.3), never a fake PASS (§11.4.69 no fail-open skip).

7. Rollout / rollback + host-install steps

7.1 Rollout (staged, gated)

1. **Phase 0 operator decisions** resolved (§9).
2. Build the custom Caddy image (DNS-01) via containers-submodule pkg/crossbuild; assert the DNS module is present.
3. Operator creates the Podman secret for the DNS token (out of band) + optional host sysctl / NAT for :443.
4. Bring up proxy-caddy against **STAGING** ACME; verify a staging cert issues + /health serves over HTTPS.
5. Run the full §6 test matrix (hermetic + staging) green with captured evidence.
6. **Cutover**: flip CADDY_ACME_CA to production; bring the service up; verify a **trusted** cert (browser-valid) + capture the chain. One-shot issuance (rate-limit aware).
7. Add the §11.4.45 docs/design/.../Status.md + Status_Summary; register the §11.4.135 guards into the standing suite; sync all doc exports.

7.2 Rollback

- The front is **additive** (§11.4.122): removing docker-compose.https.yml from the overlay set (or stopping proxy-caddy via the containers submodule) returns the system to its pre-LE state with zero impact on the proxies or the mTLS API.
- Cert/account-key loss ⇒ re-issue via ACME (the §11.4.77 regeneration mechanism); no data lost because the caddy-data volume is reproducible from ACME.
- Staging→prod is reversible by flipping CADDY_ACME_CA back to staging (though a prod cert already issued stays valid until expiry).

7.3 Host-install steps (operator-performed; documented, NOT executed by this design — CONST-033, no host network changes)

- Create the Podman secret: `podman secret create helixproxy_caddy_dns_token <token-file>` (least-privilege DNS token).
- Decide + apply the :443 reachability option (§4.4) — sysctl OR firewall NAT OR front-LB. Operator action.
- Ensure a persistent, gitignored path for the caddy-data volume.

- (No `systemd --user` timer is required in the recommended Caddy design — renewal is in-process. IF the operator instead picks `lego/certbot`, the plan's alternative branch adds a **`systemd --user timer`** — never cron-in-CI, never a git hook, per §11.4.156.)

8. Constitution compliance matrix

Rule	How this design honors it
§11.4.161 rootless Podman	proxy-caddy runs rootless; privileged-port handled via operator <code>sysctl/NAT</code> , not root
§11.4.76 containers submodule	Caddy image build (<code>pkg/crossbuild</code>), boot (<code>pkg/boot/pkg/compose</code>), readiness (<code>pkg/health</code>); no ad-hoc podman
§11.4.156 no CI/CD, no git hooks	renewal = in-process Caddy loop (long-running container), NOT cron/CI/hook. <code>lego/certbot</code> fallback uses a <code>systemd --user timer</code>
CONST-033 no host power changes	none proposed; host steps are non-power <code>sysctl/NAT</code> , operator-performed
§11.4.10 / §11.4.30 secrets	DNS token = Podman secret NAME only; ACME key + certs gitignored; leak audit before store
§11.4.74 catalogue-first	checked both orgs (<code>gh+glab</code>) → no-match; reuse published Caddy engine, no new submodule
§11.4.99 / §11.4.150 latest-source research	multi-angle research cited with URLs + 2026-07-01 access date
§11.4.169 all test types	full matrix incl. hermetic Pebble local-ACME + staging + renewal/rotation/negative
§11.4.135 / §1.1	five named guards each with a paired mutation; analyzer self-validated
§11.4.122 no silent removal	additive front; removes nothing; mTLS API + proxies untouched

9. OPERATOR INPUT REQUIRED (§11.4.66)

These decisions cannot be made autonomously — they depend on operator-owned infrastructure, DNS control, and risk tolerance for the mission-critical host. Each is framed as concrete options with a Recommended choice and what the implementer does next.

Decision A — What hostname is the public cert FOR?

Let's Encrypt issues a cert for a specific DNS name that must resolve to (or be DNS-controllable by) this host. There is no such name in the repo today.

- **[A1] A dedicated dashboard hostname** (e.g. `proxy.example.com`) — *Recommended*. Smallest surface, single SAN. Implementer sets `CADDY_DASHBOARD_DOMAIN` and issues a single-name cert.
- **[A2] A wildcard** (e.g. `*.proxy.example.com`) — only if the dashboard will grow subdomains. Requires DNS-01 (wildcards are DNS-01-only). Implementer configures a wildcard issuer.
- **[A3] Defer** — no public hostname yet; ship the whole feature against **Pebble + staging only** and leave production issuance as a later operator-gated step.

Implementer needs the exact FQDN before any staging or production issuance.

Decision B — Challenge type: DNS-01 vs HTTP-01?

- **[B1] DNS-01 with a DNS provider API token** — *Recommended*. No inbound port for validation (safest for the mission-critical host), supports wildcards, works behind NAT. **Operator must:** name the DNS provider (Cloudflare/Route53/OVH/...) and provide a least-privilege API token **as a Podman secret** (`helixproxy_caddy_dns_token`) out of band. Implementer builds the custom Caddy image with that provider's DNS module.
- **[B2] HTTP-01** — simpler, no DNS token, stock Caddy image. **Operator must confirm inbound :80 can be safely bound/reachable on this host.** Implementer skips the custom image.
- **[B3] TLS-ALPN-01** — avoids :80 but needs inbound :443. Niche; choose only if :80 is impossible but :443 inbound is fine.

This choice changes Phase 1 (custom image built only for DNS-01) and the secret handling.

Decision C — Staging-first, then production? (and who owns :443)

- **[C1] Staging → gated production cutover** — *Recommended*. Iterate entirely on LE staging (no rate-limit risk), cut to production as a one-shot after the full test matrix is green. Implementer defaults CADDY_ACME_CA to staging and treats prod as an operator-approved flip.
- **[C2] Pebble/staging only, no production yet** — pairs with [A3]; fully hermetic + staging, production deferred.
- **[C3] Straight to production** — *Not recommended* (burns rate-limit budget on iteration; risks the 50/week + 5 duplicate/week limits).

Sub-question C-port — how does :443 reach the rootless container? **[C-port-1]** operator sets `net.ipv4.ip_unprivileged_port_start=443` (host sysctl, operator-applied); **[C-port-2]** publish :8443 + operator firewall NAT 443→8443 (*Recommended* if unsure — no privileged bind); **[C-port-3]** front behind an existing host LB that already owns :443.

Sources verified (2026-07-01)

- Caddy — Automatic HTTPS: <https://caddyserver.com/docs/automatic-https>
- Caddy Community — Rootless Podman ACME/DNS: <https://caddy.community/t/rootless-podman-acme-dns-challenge-problem/33595>
- Caddy + Docker wildcard (custom DNS image required), 2026-02-08: <https://oneuptime.com/blog/post/2026-02-08-how-to-run-caddy-with-docker-and-automatic-https-wildcard-certificates/view>
- Wildcard TLS with Caddy + Cloudflare, 2026 (HostMyCode): <https://www.hostmycode.com/blog/linux-vps-acme-dns-challenge-automation-wildcard-tls-caddy-cloudflare-2026>
- Lego — ACME client & library (Go): <https://go-acme.github.io/lego/>
- ACME clients (certbot/lego/acme.sh) on Ubuntu, 2026-03-02: <https://oneuptime.com/blog/post/2026-03-02-use-acme-clients-certbot-lego-acmesh-ubuntu/view>
- certbot vs acme.sh, 2026 (SSLInsights): <https://sslinsights.com/certbot-vs-acme-sh/>
- Let's Encrypt — Rate Limits: <https://letsencrypt.org/docs/rate-limits/>
- Let's Encrypt — Shorter lifetimes & rate limits (90→64→45d, ARI), 2026-02-24: <https://letsencrypt.org/2026/02/24/rate-limits-45-day-certs>
- Let's Encrypt — Staging Environment: <https://letsencrypt.org/docs/staging-environment/>

- Pebble (letsencrypt/pebble) — miniature ACME test server: <https://github.com/letsencrypt/pebble>
- How Pebble supports ACME client developers (LE, 2025-04-30): <https://letsencrypt.org/2025/04/30/pebbleacmeimplementation>
- Run your own private ACME server with step-ca (Smallstep): <https://smallstep.com/blog/private-acme-server/>

Negative findings (per §11.4.99 honesty): the fetched Caddy automatic-HTTPS page does **not** state an exact renewal-window day count, and does **not** mention ARI by name; the design therefore does not hard-code a renewal day count and instead verifies the renewal behaviour empirically in the renewal-simulation test. The LE 2026 page confirms renewals are rate-limit-exempt but did not, on the fetched view, spell out ARI mechanics — treat ARI as "supported by modern Caddy/CertMagic" and verify at implementation time against the then-current Caddy release notes (re-verify before prod cutover per §11.4.99 90-day staleness for risk-classified services).